

# 17-03-2025

## Les structures

Les premiers types déjà vu en cours

- char → octet - 1 octet
- int - plus d'octets que char
- char \* → pointeur vers un caractère
- unsigned char → un octet n'ayant pas de byte de signe
- `size_t` → contient une longueur - n'est pas signé - le nombre d'octet est inconnu

On ne sait pas pour la plupart de ces types là (et donc autre que char) , combien d'octet ils contiennent

---

[open-std.org/jtc1/sc22/wg14/www/docs/n3220.pdf](https://open-std.org/jtc1/sc22/wg14/www/docs/n3220.pdf)

C24 - à jour officiel

Le C est un langage toujours en évolution et la rétrocompatibilité ne fonctionne pas toujours

Détail : c'est fait par l'ISO du gouvernement américain

Regarder le brouillon car la release est payante mais la différence n'est pas grand chose - cette fois-ci la différence est une faute d'orthographe

---

Aujourd'hui on va créer des nouveaux types

`struct` pour créer un nouveau type

**C** c

```
1 struct point {
2     int x;
3     int y;
4 }; // Ce code crée la structure x et y et les met côte à côte
```

`typedef` permet de renommer un type

**C** c

```
1 int main (void) {
2     int toto;
```

```
3     struct point p1;
4
5     p1.x = 10;
6     p1.y = 11;
7
8     return 0;
9 }
```

---

Le but de `malloc` est de gérer la heap

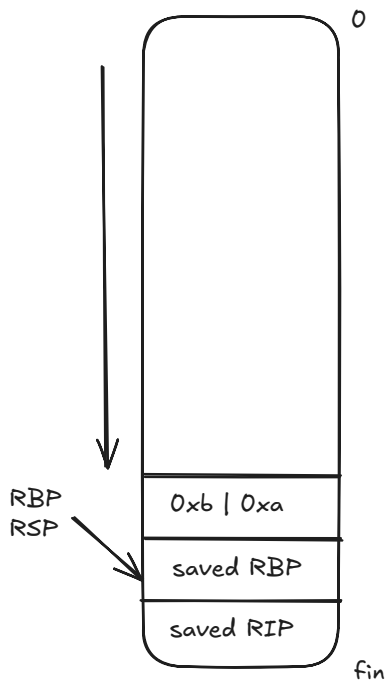
---

```
cc -std=c99 -O2 -pedantic -pedantic-errors -Wall -Wbad-function-cast -Wcast-align=strict -Wcast-qual -Werror -Wextra -Wfloat-equal -Wformat=2 -Winit-self -Winline -Wlogical-op -Wnested-externs -Wredundant-decls -Wshadow -Wstrict-aliasing -Wstrict-overflow=3 -Wstrict-prototypes -Wundef -Wuninitialized -Wwrite-strings -fanalyzer hello.c -o hello → flag de compilation
```

```
gcc -c struct.c -c struct → -c permet de sauvegarder les lignes
```

#### Asm

```
1  Dump of assembleur code for function main:
2  0x000... <+0>  push  %rbp
3  0x000... <+1>  mov   %rsp, %rbp
4  0x000... <+4>  movl  $0xa, -0x8(%rbp)
5  0x000... <+11> movl  $0xb, -0x4(%rbp)
6  0x000... <+18> mov   $0x0, %eax
7  0x000... <+23> pop   %rbp
8  0x000... <+24> ret
9  End of assembleur dump
```



On veut que le fonctionne agrandise la partie utilise de la stack pour ce code(?)

**C** c

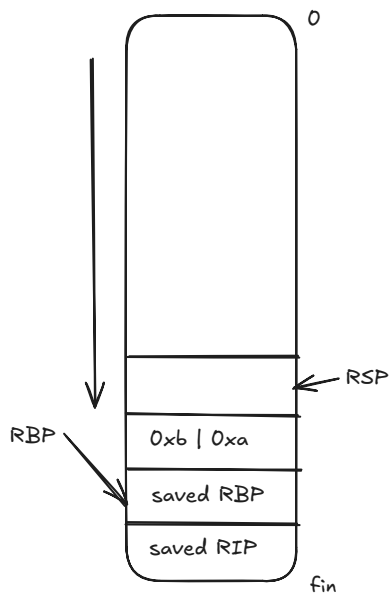
```

1  int write(int, char *, int); //pour savoir dans quel registre il faut ele
   mettre
2
3  int main (void) {
4      int toto;
5      struct point p1;
6
7      p1.x = 10;
8      p1.y = 11;
9
10     write(1, "hello\n", 6);
11
12     return 0;
13 }
```

### Asm

```

1  Dump of assembleur code for function main:
2  0x000... <+0>   push    %rbp
3  0x000... <+1>   mov     %rsp, %rbp
4  0x000... <+4>   sub     $0x10, %rsp # Cette ligne montre que la taille est
5                                     agradit pour pouvoir mettre rsp dans
6                                     la
7                                     stack
8                                     -> int toto et struct point p1
8  ...
```



On voit que la ligne prends 16 octets mais n'en utilise que 12, pourquoi ?

Il arrondit 8 octet à 64 bits pour que la fonction `push rbp` de la ligne suivant ne soit pas séparé dans la stack pour qu'elle soit aligné et donc que le code soit plus rapide

→ Il range tout correctement

Il y a un saying que pour le C il faut gérer la mémoire soit même, mais ce n'est pas totalement vrai car pour l'instant dans la stack, elle se gère elle-même mais pour la heap il faudra vraiment gérer la mémoire soit même

Pour le `include` les `<>` cherche les bibliothèque dans le repertoire système et les `""` cherche dans les bibliothèque locales

***Est-ce que les includes peuvent alourdir le projet ?***

Non, car l'include montre juste les registres à utiliser

Le `#include <>` fait juste un copier coller de la bibliothèque dans notre code

**C** c

```
1  #include <stdlib.h>
2  #include <sqrt.h>
3  #include <stdio.h>
4
5  struct point {
6      int x;
```

```

7     int y;
8 }
9
10 int write(int, char *, int);
11
12 double point_dist_from_point(struct point self, struct point other) {
13     return sqrt(
14         pow(self.x - other.x, 2) +
15         pow(self.y - other.y, 2)
16     );
17 }
18
19 double point_dist_from_origin(struct point self) {
20     struct point origin;
21
22     origin.x = 0
23     origin.y = 0
24
25     return point_dist_from_point(self, origin);
26
27 void point_print(struct point self) {
28     printf("Point(x=%d, y=%d)\n", self.x, self.y))
29 }
30
31 int main (void) {
32     int toto;
33     struct point p1;
34
35     p1.x = 10;
36     p1.y = 11;
37
38     point_print(p1);
39     return 0;
40 }

```

Les adresses virtuelle dans la RAM de début et de fin ont tous un 000 à la fin car se sont des gros blocks pour pouvoir mettre les permissions

tips : utiliser `strace` pour débbugger

**C**

c

```

1 #include <stdlib.h>
2 #include <sqrt.h>
3 #include <stdio.h>
4 #include <unistd.h>
5

```

```

6  struct png {
7      // 8 octets pour la signature
8      uint64_t data_length;
9      uint32_t chunk_name;
10     uint32_t width;
11     uint32_t height;
12     unsigned char bit_dept;
13     unsigned char color_type;
14     unsigned char compression;
15     unsigned char filter;
16     unsigned char interleaving;
17 }
18
19 struct point {
20     int x;
21     int y;
22 }
23
24 int write(int, char *, int);
25
26 double point_dist_from_point(struct point self, struct point other) {
27     return sqrt(
28         pow(self.x - other.x, 2) +
29         pow(self.y - other.y, 2)
30     );
31 }
32
33 double point_dist_from_origin(struct point self) {
34     struct point origin;
35
36     origin.x = 0
37     origin.y = 0
38
39     return point_dist_from_point(self, origin);
40
41 void point_print(struct point self) {
42     printf("Point(x=%d, y =%d)\n", self.x, self.y))
43 }
44
45 int main (void) {
46     struct png png_data;
47
48     int open fd = open("test.png", 0_RDONLY); //int car on peu avoir -1
49
50     if (fd == -1) { // fd = File Descriptor -> numéro de fichier
51         printf("Can't open test.png\n");
52         return EXIT_FAILURE;
53     }
54

```

```
55     read(fd, png_data, sizeof(png_data));
56
57     return EXIT_SUCCESS;
```

Pourquoi `png_datawidth` et `png_data.height` valent autant ?

C

c

```
1  #include <endian.h>
2  #include <stdlib.h>
3  #include <math.h>
4  #include <stdio.h>
5  #include <unistd.h>
6  #include <stdint.h>
7  #include <fcntl.h>
8
9  struct png {
10     uint64_t sig;
11     uint32_t data_length;
12     uint32_t chunk_name;
13     uint32_t width;
14     uint32_t height;
15     unsigned char bit_depth;
16     unsigned char color_type;
17     unsigned char compression;
18     unsigned char filter;
19     unsigned char interleaving;
20 };
21
22 struct point {
23     long x;
24     long y;
25 };
26
27 double point_dist_from_point(struct point self, struct point other) {
28     return sqrt(
29         pow(self.x - other.x, 2) +
30         pow(self.y - other.y, 2)
31     );
32 }
33
34 double point_dist_from_origin(struct point self) {
35     struct point origin;
36
37     origin.x = 0;
38     origin.y = 0;
```

```

39
40     return point_dist_from_point(self, origin);
41 }
42
43 void point_print(struct point self) {
44     printf("Point(x=%ld, y=%ld)\n", self.x, self.y);
45 }
46
47 int main(void) {
48     struct png png_data;
49     // File Descriptor (numéro du fichier)
50     int fd = open("test.png", O_RDONLY);
51
52     if (fd == -1) {
53         printf("Can't open test.png\n");
54         return EXIT_FAILURE;
55     }
56
57     read(fd, &png_data, sizeof(png_data));
58
59     png_data.height = be32toh(png_data.height);
60     png_data.width = be32toh(png_data.width);
61
62     printf("%s: %d x %d\n", "test.png", png_data.width, png_data.height);
63     return EXIT_SUCCESS;
64 }

```

be32toh conversion de big endian à host endian

### Pourquoi doit-on convertir de big endian à H endian ?

Car nos ordinateurs ne sont pas toujours en big endian et donc H endian → endian de l'Host et donc de l'ordi

### Quel est la différence entre big et little endian ?

